coursera | UNIVERSITY OF MICHIGAN

Applied Information Extraction in Python

Today's goal • 3/3 items

Module 1
Information Extraction

Module 2
Named Entity Recognition (NER)

Module 3
Neural Network Models

Module 4
Transformers Transform Information Extraction

Language Models

- Language Models (LMs)
  Video • 11 min
- Large Language Models (LLMs)
  Video • 7 min
- Knowledge Check: Language Models
  Practice Assignment • Grade: 100%

What Are Transformers?

- Transformers
  Video • 9 min
- Recent Advances in GPTs
  Reading • 10 min
- Knowledge Check: What Are Transformers?
  Practice Assignment • Grade: 100%

How to Build Information Extraction Models Using Transformers

- **Building an Information Extraction Pipeline with Transformers and LLMs**
  Reading • 10 min
- Design an Information Extraction System for Sports News Reports
  Dialogue • 15 min

Module Wrap-Up & Assignment

- Assignment 4 Introduction: Build an Information Extraction Pipeline using Transformers
  Video • 4 min
- Build an Information Extraction Pipeline using Transformers
  Programming Assignment • 3h
- Module 4 Assignment
  Graded Assignment • 30 min

Course Wrap Up

- Course Wrap-Up
  Video • 4 min
- Continue Your Journey and Earn a Master of Applied Data Science Degree Online
  Reading • 10 min
- Course Post-Survey
  Reading • 10 min

Building an Information Extraction Pipeline with Transformers and LLMs

0:22 / 4:13 2x

Welcome to Modern NLP

You've now reached the most advanced part of this course. This is where traditional models step aside and **transformers** take the lead, not just identifying words, but understanding meaning at scale.

In this reading, we'll walk through:

- What language models and LLMs are
- Why transformers became dominant
- How pre-trained models are fine-tuned for IE tasks
- The practical steps needed to use them in your own pipeline

What Are Language Models (LM)?

At their core, **language models** predict the next word in a sequence based on prior words.

Example:

"He picked up his..." → likely completion? "Spatula" is more probable than "Velvet."

A language model ranks completions based on how often and how naturally they appear in real-world language.

From N-gram to Neural Nets

Model Type	Description	Limitation
Unigram	No history; word frequency only	Ignores context ("the the of and")
Bigram	One-word history	Limited context
Trigram	Two-word history	Better, still narrow
N-gram (n > 3)	Longer context	Sparse data; no generalization
Neural LMs	Learned embeddings	More flexible
Transformers	Attention over full sequence	State-of-the-art

Key insight: More context → more meaningful predictions.

What Are Large Language Models (LLMs)?

LLMs are large-scale neural language models trained on massive datasets, often with **billions of parameters**.

They go beyond predicting words, they can:

- Answer questions
- Translate languages
- Perform arithmetic
- Fill in blanks using world knowledge
- Handle **zero-shot tasks** without retraining

These skills emerge only when models are trained at **massive scale**, a phenomenon known as **emergent ability**.

What Are Transformers?

Transformers are deep neural architectures that replaced recurrence (like in LSTMs) with **self-attention**.

They consist of:

- **Encoder**: Turns text into embeddings
- **Decoder**: Generates output (e.g., predictions or text)
- **Attention Layers**: Learn which words are most important

Transformers allow for:

- **Parallel training** → faster, scalable
- **Global context** → see the entire sentence, not just the previous word
- **Transfer learning** → pre-train once, fine-tune for many tasks

Preprocessing for Transformer Pipelines

Before you fine-tune a model like BERT, you must:

1. **Tokenize the text** using a model-specific tokenizer
2. Add special tokens like [CLS] (start) and [SEP] (separator)
3. **Map tokens to IDs** from the model's vocabulary
4. **Pad sequences** to a consistent length
5. Generate **attention masks** to ignore padding during training

Example:

Input: *"University of Michigan"*

→ Tokens: [CLS], university, of, michigan, [SEP]

→ IDs: [101, 2030, 1997, 2637, 102]

Fine-Tuning Transformers for Information Extraction

You can now use models like **bert-base-cased** for IE tasks like NER.

Steps:

- Load a **pre-trained transformer model**
- Add a **classification head** (for token-level prediction)
- Fine-tune on your labeled dataset

Training uses:

- **Cross-entropy loss** for token classification
- **AdamW optimizer**
- A learning rate **scheduler** for better convergence

Even small labeled datasets can produce strong results, thanks to pre-training.

Evaluation & Output

Once trained, the model predicts structured outputs like BIO tags.

Example:

Input: *"123 Main Street, Ann Arbor"*

Output: [B-ADDR, I-ADDR, I-ADDR, B-CITY, I-CITY]

Metrics:

- **Token-level accuracy**
- **F1 score** for entity spans
- **Span overlap** vs. gold standard labels

Why Transformers Work So Well

- **Learned Representations**: Capture both meaning and structure
- **Transfer Learning**: Pre-trained on general corpora, fine-tuned for your task
- **Emergent Abilities**: Reasoning, domain knowledge, multilingual support

They support:

- Zero-shot learning
- Few-shot learning
- Domain-specific variants (e.g., BioBERT, ClinicalBERT)

Suggested Resources

- **Hugging Face Course – Fine-Tuning BERT for NER**
- **Google's Original BERT Paper**

Reflection Prompt

You've worked with CRFs, LSTMs, and now transformers.

Why do LLMs like ChatGPT often outperform older models in information extraction, even without task-specific training? What trade-offs do transformers introduce (e.g., compute requirements vs. performance)?

Go to next item ✓ Completed

Like Dislike Report an issue

Go to next item →